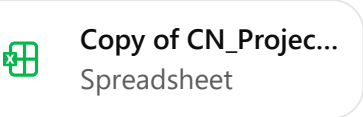
You said:



i have a course's (computer networks) project list to choose one of them and i need you to tell whether it is a good choice or not for learning computer networks hands-on,i had done project 3 already and i want to choose another

Worked for 1m 4s

Analyzed

Analyzed

Analyzed

Analyzed

ChatGPT said:

‹/›

You said:

i have a laptop with WSL installed ,can i do project 6?

Worked for 1m 15s

Analyzed

Analyzed

Analyzed

ChatGPT said:

‹/›

You said:

so which project do you think better and it involves networking,OS and using hands-on tools like wireshark and not complicated

Worked for 50s

ChatGPT said:

You said:

project 11?

Worked for 55s

ChatGPT said:

You said:

let's start doing project 11 from scratch

Worked for 1m 3s

ChatGPT said:

You said:

A-D are successful,wireshark is installed already

Worked for 23s

ChatGPT said:

You said:

1->yes
2->yes

Worked for 34s

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/inspect_tls.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 21

--- Summary ---
ClientHello messages parsed: 0
ServerHello messages parsed: 0
One or both handshake types were not parsed. If Wireshark

displays them, tell me; we can investigate Scapy's packet decoding or TCP reassembly.

Worked for 57s >

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/inspect_tls.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 21

ClientHello #1
  Source:             172.23.55.87
  Destination:        104.20.23.154
  Legacy version:     771
  Cipher suites:      [4866, 4867, 4865, 49196, 49200, 159, 52393,
52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191,
103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47,
255]
  Extension IDs:      [0, 11, 10, 16, 22, 23, 49, 13, 43, 45, 51, 21]
  Supported groups:   [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
  EC point formats:   [0, 1, 2]

ServerHello #1
  Source:             104.20.23.154
  Destination:        172.23.55.87
  Legacy version:     771
  Selected cipher:    4866
  Extension IDs:      [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$
```

Worked for 38s >

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 21

ClientHello #1
  Source:       172.23.55.87
  Destination:  104.20.23.154
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-16-22-23-49-13-43-45-51-21,29-23-30-25-24-256-257-258-
259-260,0-1-2
  JA3 MD5:      0149f47eabf9a20d0893e2a44e5a6323

ClientHello messages processed: 1
```

Worked for 27s >

Analyzed

ChatGPT said:

‹/›

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3s.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 21

ServerHello #1
  Source:       104.20.23.154
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
  JA3S MD5:     907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1\
```

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python -m unittest discover -s tests -t . -v
test_grease_values_are_removed
(tests.test_fingerprints.FingerprintTests.test_grease_values_are_removed) ... ok
test_ja3_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3_fingerprint) ... ok
test_ja3s_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3s_fingerprint) ... ok
test_md5_known_value
(tests.test_fingerprints.FingerprintTests.test_md5_known_value) ... ok

---
Ran 4 tests in 0.055s

OK
before proceeding we can't you explain the codes which we have written

ChatGPT said:

You said:

let's do it

Worked for 23s

ChatGPT said:

You said:

1 selection

i can't run that command on terminal

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2

```
$ python check_reassembly.py
Capture: captures/first_https.pcap
Packets returned by Scapy: 16
ClientHello messages: 1
ServerHello messages: 1
```

Worked for 19s >

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python -m unittest discover -s tests -t . -v
test_grease_values_are_removed
(tests.test_fingerprints.FingerprintTests.test_grease_values_are_remo
ved) ... ok
test_ja3_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3_fingerprint) ... ok
test_ja3s_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3s_fingerprint) ... ok
test_md5_known_value
(tests.test_fingerprints.FingerprintTests.test_md5_known_value) ... ok
test_published_ja3_reference_vector
(tests.test_fingerprints.FingerprintTests.test_published_ja3_reference
_vector) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.025s

OK
```

Worked for 20s >

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python -m unittest discover -s tests -t . -v
test_grease_values_are_removed
(tests.test_fingerprints.FingerprintTests.test_grease_values_are_remo
ved) ... ok
```

```
test_ja3_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3_fingerprint) ... ok
test_ja3s_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3s_fingerprint) ... ok
test_md5_known_value
(tests.test_fingerprints.FingerprintTests.test_md5_known_value) ... ok
test_published_ja3_reference_vector
(tests.test_fingerprints.FingerprintTests.test_published_ja3_reference
_vector) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.016s

OK
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 16

ClientHello #1
  Source:       172.23.55.87
  Destination:  104.20.23.154
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-16-22-23-49-13-43-45-51-21,29-23-30-25-24-256-257-258-
259-260,0-1-2
  JA3 MD5:      0149f47eabf9a20d0893e2a44e5a6323

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3s.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 16

ServerHello #1
  Source:       104.20.23.154
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
  JA3S MD5:     907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1
(.venv)
```

busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$

Worked for 4m 50s >

ChatGPT said:

You said:

```
import hashlib
import unittest
from pathlib import Path

from scapy.all import load_layer, rdpcap
from types import SimpleNamespace
# Enable Scapy's TLS parser before importing TLS classes.
load_layer("tls")

from scapy.layers.tls.handshake import TLSClientHello,
TLSServerHello

from src.ja3 import calculate_ja3, without_grease
from src.ja3s import calculate_ja3s


PROJECT_ROOT = Path(__file__).resolve().parents[1]
PCAP_FILE = PROJECT_ROOT / "captures" / "first_https.pcap"


class FingerprintTests(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        if not PCAP_FILE.exists():
            raise FileNotFoundError(f"Capture not found: {PCAP_FILE}")

        packets = rdpcap(str(PCAP_FILE))

        cls.client_hello = next(
            (
                packet[TLSClientHello]
                for packet in packets
                if packet.haslayer(TLSClientHello)
            ),
            None,
```

```
        )

        cls.server_hello = next(
            (
                packet[TLSServerHello]
                for packet in packets
                if packet.haslayer(TLSServerHello)
            ),
            None,
        )

        if cls.client_hello is None or cls.server_hello is None:
            raise AssertionError(
                "The capture must contain both TLS handshake
messages."
            )

    def test_grease_values_are_removed(self):
        values = [0x0A0A, 4866, 0x1A1A, 47, 0xFAFA]

        result = without_grease(values)

        self.assertEqual(result, [4866, 47])

    def test_ja3_fingerprint(self):
        ja3_string, fingerprint = calculate_ja3(self.client_hello)

        self.assertEqual(
            ja3_string,
            "771,4866-4867-4865-49196-49200-159-"
            "52393-52392-52394-49195-49199-158-"
            "49188-49192-107-49187-49191-103-"
            "49162-49172-57-49161-49171-51-157-"
            "156-61-60-53-47-255,"
            "0-11-10-16-22-23-49-13-43-45-51-21,"
            "29-23-30-25-24-256-257-258-259-260,"
            "0-1-2",
        )

        self.assertEqual(
            fingerprint,
            "0149f47eabf9a20d0893e2a44e5a6323",
        )

    def test_ja3s_fingerprint(self):
```

```
        ja3s_string, fingerprint = calculate_ja3s(self.server_hello)

        self.assertEqual(ja3s_string, "771,4866,51-43")

        self.assertEqual(
            fingerprint,
            "907bf3ecef1c987c889946b737b43de8",
        )

    def test_md5_known_value(self):
        # Verify Python's MD5 operation against a standard test value.
        self.assertEqual(
            hashlib.md5(b"hello").hexdigest(),
            "5d41402abc4b2a76b9719d911017c592",
        )
    def test_published_ja3_reference_vector(self):
        # Create a small object representing the published JA3
example.
        # SimpleNamespace lets us supply the fields expected by our
function.
        hello = SimpleNamespace(
            version=769,
            ciphers=[
                47, 53, 5, 10, 49161, 49162,
                49171, 49172, 50, 56, 19, 4,
            ],
            ext=[
                SimpleNamespace(type=0),
                SimpleNamespace(type=10, groups=[23, 24, 25]),
                SimpleNamespace(type=11, ecpl=[0]),
            ],
        )

        ja3_string, fingerprint = calculate_ja3(hello)

        expected_string = (
            "769,"
            "47-53-5-10-49161-49162-49171-49172-50-56-19-4,"
            "0-10-11,"
            "23-24-25,"
            "0"
        )

        expected_fingerprint =
"ada70206e40642a3e4461f35503241d5"
```

```
        self.assertEqual(ja3_string, expected_string)
        self.assertEqual(fingerprint, expected_fingerprint)

if __name__ == "__main__":
    unittest.main()
```

this is the test_fingerprints.py file,once see all the conversion history,you might be missing information

Worked for 4m 36s >

ChatGPT said:

You said:

```
#!/usr/bin/env python3
"""Inspect TLS ClientHello and ServerHello messages in a PCAP file."""

import argparse

from scapy.all import IP, IPv6, rdpcap,load_layer
from scapy.layers.tls.handshake import TLSClientHello,
TLSServerHello

load_layer("tls")

def get_endpoints(packet):
    """Return source and destination IP addresses."""
    if IP in packet:
        return packet[IP].src, packet[IP].dst

    if IPv6 in packet:
        return packet[IPv6].src, packet[IPv6].dst

    return "unknown", "unknown"


def get_extension_ids(hello):
    """Return extension type numbers in their original order."""
    extensions = getattr(hello, "ext", None) or []

    return [
```

```
            int(extension.type)
            for extension in extensions
            if getattr(extension, "type", None) is not None
        ]


    def inspect_client_hello(packet, hello, number):
        """Print the fields needed to understand a JA3 fingerprint."""
        source, destination = get_endpoints(packet)

        ciphers = [
            int(cipher)
            for cipher in (getattr(hello, "ciphers", None) or [])
        ]

        extensions = getattr(hello, "ext", None) or []
        extension_ids = get_extension_ids(hello)

        supported_groups = []
        point_formats = []

        for extension in extensions:
            extension_type = getattr(extension, "type", None)

            # Extension 10: supported groups (formerly elliptic curves).
            if extension_type == 10:
                supported_groups.extend(
                    int(group)
                    for group in (getattr(extension, "groups", None) or [])
                )

            # Extension 11: supported elliptic-curve point formats.
            elif extension_type == 11:
                point_formats.extend(
                    int(value)
                    for value in (getattr(extension, "ecpl", None) or [])
                )

        print(f"\nClientHello #{number}")
        print(f"  Source:              {source}")
        print(f"  Destination:         {destination}")
        print(f"  Legacy version:      {hello.version}")
        print(f"  Cipher suites:       {ciphers}")
        print(f"  Extension IDs:       {extension_ids}")
        print(f"  Supported groups:    {supported_groups}")
```

```
    print(f"  EC point formats:    {point_formats}")


def inspect_server_hello(packet, hello, number):
    """Print basic ServerHello fields for later JA3S work."""
    source, destination = get_endpoints(packet)

    print(f"\nServerHello #{number}")
    print(f"  Source:              {source}")
    print(f"  Destination:         {destination}")
    print(f"  Legacy version:      {hello.version}")
    print(f"  Selected cipher:     {hello.cipher}")
    print(f"  Extension IDs:       {get_extension_ids(hello)}")


def main():
    parser = argparse.ArgumentParser(
        description="Inspect TLS handshakes in a PCAP file."
    )
    parser.add_argument(
        "pcap",
        nargs="?",
        default="captures/first_https.pcap",
        help="Path to the PCAP file",
    )
    args = parser.parse_args()

    packets = rdpcap(args.pcap)

    client_count = 0
    server_count = 0

    print(f"Capture file: {args.pcap}")
    print(f"Total packets: {len(packets)}")

    for packet in packets:
        if packet.haslayer(TLSClientHello):
            client_count += 1
            inspect_client_hello(
                packet, packet[TLSClientHello], client_count
            )

        if packet.haslayer(TLSServerHello):
            server_count += 1
            inspect_server_hello(
```

```
                packet, packet[TLSServerHello], server_count
            )

    print("\n--- Summary ---")
    print(f"ClientHello messages parsed: {client_count}")
    print(f"ServerHello messages parsed: {server_count}")

    if client_count == 0 or server_count == 0:
        print(
            "One or both handshake types were not parsed. "
            "If Wireshark displays them, tell me; we can investigate "
            "Scapy's packet decoding or TCP reassembly."
        )


if __name__ == "__main__":
    main()
```

above code is inspect_tls.py

```
#!/usr/bin/env python3
"""Calculate JA3 fingerprints from TLS ClientHello messages in a
PCAP."""

import argparse
import hashlib
from scapy.all import IP, IPv6, load_layer, sniff
from scapy.sessions import TCPSession

# Enable Scapy's TLS packet dissector.
load_layer("tls")

from scapy.layers.tls.handshake import TLSClientHello


# GREASE values defined by RFC 8701.
# These reserved values help TLS implementations exercise
extensibility.
GREASE_VALUES = {
    0x0A0A, 0x1A1A, 0x2A2A, 0x3A3A,
    0x4A4A, 0x5A5A, 0x6A6A, 0x7A7A,
    0x8A8A, 0x9A9A, 0xAAAA, 0xBABA,
    0xCACA, 0xDADA, 0xEAEA, 0xFAFA,
}
```

```
def get_endpoints(packet):
    """Return the packet's source and destination IP addresses."""
    if IP in packet:
        return packet[IP].src, packet[IP].dst

    if IPv6 in packet:
        return packet[IPv6].src, packet[IPv6].dst

    return "unknown", "unknown"


def without_grease(values):
    """Remove GREASE values while preserving the remaining
order."""
    return [
        int(value)
        for value in values
        if int(value) not in GREASE_VALUES
    ]


def hyphen_join(values):
    """Represent integer values as a hyphen-separated string."""
    return "-".join(str(value) for value in values)


def build_ja3_string(hello):
    """Build the five comma-separated fields used by JA3."""

    # Field 1: ClientHello legacy version.
    version = int(hello.version)

    # Field 2: Cipher suites, with GREASE values removed.
    ciphers = without_grease(
        getattr(hello, "ciphers", None) or []
    )

    extensions = getattr(hello, "ext", None) or []

    # Field 3: Extension IDs, preserving their original order.
    extension_ids = without_grease(
        [
            int(extension.type)
            for extension in extensions
```

```
            if getattr(extension, "type", None) is not None
        ]
    )

    supported_groups = []
    point_formats = []

    for extension in extensions:
        extension_type = getattr(extension, "type", None)

        # Field 4: Supported groups from extension type 10.
        if extension_type == 10:
            supported_groups = without_grease(
                getattr(extension, "groups", None) or []
            )

        # Field 5: EC point formats from extension type 11.
        elif extension_type == 11:
            point_formats = [
                int(value)
                for value in (
                    getattr(extension, "ecpl", None) or []
                )
            ]

    # JA3 requires five fields separated by commas.
    # Lists within each field are separated by hyphens.
    ja3_parts = [
        str(version),
        hyphen_join(ciphers),
        hyphen_join(extension_ids),
        hyphen_join(supported_groups),
        hyphen_join(point_formats),
    ]

    return ",".join(ja3_parts)


def calculate_ja3(hello):
    """Return the JA3 string and its hexadecimal MD5 digest."""
    ja3_string = build_ja3_string(hello)

    # MD5 is used for compatibility with the JA3 fingerprint format,
    # not as a security mechanism.
    fingerprint = hashlib.md5(
```

```
        ja3_string.encode("ascii")
    ).hexdigest()

    return ja3_string, fingerprint


def main():
    parser = argparse.ArgumentParser(
        description="Calculate JA3 fingerprints from a PCAP."
    )

    parser.add_argument(
        "pcap",
        nargs="?",
        default="captures/first_https.pcap",
        help="Path to a PCAP file",
    )

    args = parser.parse_args()
    packets = sniff(
    offline=args.pcap,
    session=TCPSession,
    store=True,
)

    print(f"Capture file: {args.pcap}")
    print(f"Total packets: {len(packets)}")

    hello_count = 0

    for packet in packets:
        if not packet.haslayer(TLSClientHello):
            continue

        hello_count += 1
        source, destination = get_endpoints(packet)

        ja3_string, fingerprint = calculate_ja3(
            packet[TLSClientHello]
        )

        print(f"\nClientHello #{hello_count}")
        print(f"  Source:       {source}")
        print(f"  Destination:  {destination}")
        print(f"  JA3 string:   {ja3_string}")
```

```
        print(f"  JA3 MD5:      {fingerprint}")

    print(f"\nClientHello messages processed: {hello_count}")

    if hello_count == 0:
        print(
            "No ClientHello messages were parsed. "
            "Check the PCAP and TCP reassembly if necessary."
        )


if __name__ == "__main__":
    main()
```

above code is ja3.py

```
#!/usr/bin/env python3
"""Calculate JA3S fingerprints from TLS ServerHello messages."""

import argparse
import hashlib
from scapy.all import IP, IPv6, load_layer, sniff
from scapy.sessions import TCPSession

# Explicitly enable Scapy's TLS dissector.
load_layer("tls")

from scapy.layers.tls.handshake import TLSServerHello


GREASE_VALUES = {
    0x0A0A, 0x1A1A, 0x2A2A, 0x3A3A,
    0x4A4A, 0x5A5A, 0x6A6A, 0x7A7A,
    0x8A8A, 0x9A9A, 0xAAAA, 0xBABA,
    0xCACA, 0xDADA, 0xEAEA, 0xFAFA,
}


def get_endpoints(packet):
    """Return source and destination IP addresses."""

    if IP in packet:
        return packet[IP].src, packet[IP].dst

    if IPv6 in packet:
```

```
            return packet[IPv6].src, packet[IPv6].dst

    return "unknown", "unknown"


def without_grease(values):
    """Remove GREASE values without changing the remaining
order."""

    return [
        int(value)
        for value in values
        if int(value) not in GREASE_VALUES
    ]


def build_ja3s_string(hello):
    """Build the JA3S string from a TLS ServerHello."""

    # Field 1: ServerHello legacy version.
    version = int(hello.version)

    # Field 2: The single cipher suite selected by the server.
    cipher = int(hello.cipher)

    # Field 3: ServerHello extension IDs, in their original order.
    extensions = getattr(hello, "ext", None) or []

    extension_ids = without_grease([
        int(extension.type)
        for extension in extensions
        if getattr(extension, "type", None) is not None
    ])

    extension_string = "-".join(
        str(value) for value in extension_ids
    )

    return f"{version},{cipher},{extension_string}"


def calculate_ja3s(hello):
    """Return the JA3S string and its MD5 digest."""

    ja3s_string = build_ja3s_string(hello)
```

```
    fingerprint = hashlib.md5(
        ja3s_string.encode("ascii")
    ).hexdigest()

    return ja3s_string, fingerprint


def main():
    parser = argparse.ArgumentParser(
        description="Calculate JA3S fingerprints from a PCAP."
    )

    parser.add_argument(
        "pcap",
        nargs="?",
        default="captures/first_https.pcap",
        help="Path to a PCAP file",
    )

    args = parser.parse_args()
    packets = sniff(
    offline=args.pcap,
    session=TCPSession,
    store=True,
)

    print(f"Capture file: {args.pcap}")
    print(f"Total packets: {len(packets)}")

    hello_count = 0

    for packet in packets:
        if not packet.haslayer(TLSServerHello):
            continue

        hello_count += 1
        source, destination = get_endpoints(packet)

        ja3s_string, fingerprint = calculate_ja3s(
            packet[TLSServerHello]
        )

        print(f"\nServerHello #{hello_count}")
        print(f"  Source:       {source}")
```

```
            print(f"  Destination:  {destination}")
            print(f"  JA3S string:  {ja3s_string}")
            print(f"  JA3S MD5:     {fingerprint}")

        print(f"\nServerHello messages processed: {hello_count}")

        if hello_count == 0:
            print("No ServerHello messages were parsed.")


if __name__ == "__main__":
    main()
```

above code is ja3s.py

```
import hashlib
import unittest
from pathlib import Path

from scapy.all import load_layer, rdpcap
from types import SimpleNamespace
# Enable Scapy's TLS parser before importing TLS classes.
load_layer("tls")

from scapy.layers.tls.handshake import TLSClientHello,
TLSServerHello

from src.ja3 import calculate_ja3, without_grease
from src.ja3s import calculate_ja3s


PROJECT_ROOT = Path(__file__).resolve().parents[1]
PCAP_FILE = PROJECT_ROOT / "captures" / "first_https.pcap"


class FingerprintTests(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        if not PCAP_FILE.exists():
            raise FileNotFoundError(f"Capture not found: {PCAP_FILE}")

        packets = rdpcap(str(PCAP_FILE))

        cls.client_hello = next(
```

```
            (
                packet[TLSClientHello]
                for packet in packets
                if packet.haslayer(TLSClientHello)
            ),
            None,
        )

        cls.server_hello = next(
            (
                packet[TLSServerHello]
                for packet in packets
                if packet.haslayer(TLSServerHello)
            ),
            None,
        )

        if cls.client_hello is None or cls.server_hello is None:
            raise AssertionError(
                "The capture must contain both TLS handshake
messages."
            )

    def test_grease_values_are_removed(self):
        values = [0x0A0A, 4866, 0x1A1A, 47, 0xFAFA]

        result = without_grease(values)

        self.assertEqual(result, [4866, 47])

    def test_ja3_fingerprint(self):
        ja3_string, fingerprint = calculate_ja3(self.client_hello)

        self.assertEqual(
            ja3_string,
            "771,4866-4867-4865-49196-49200-159-"
            "52393-52392-52394-49195-49199-158-"
            "49188-49192-107-49187-49191-103-"
            "49162-49172-57-49161-49171-51-157-"
            "156-61-60-53-47-255,"
            "0-11-10-16-22-23-49-13-43-45-51-21,"
            "29-23-30-25-24-256-257-258-259-260,"
            "0-1-2",
        )
```

```
        self.assertEqual(
            fingerprint,
            "0149f47eabf9a20d0893e2a44e5a6323",
        )

    def test_ja3s_fingerprint(self):
        ja3s_string, fingerprint = calculate_ja3s(self.server_hello)

        self.assertEqual(ja3s_string, "771,4866,51-43")

        self.assertEqual(
            fingerprint,
            "907bf3ecef1c987c889946b737b43de8",
        )

    def test_md5_known_value(self):
        # Verify Python's MD5 operation against a standard test value.
        self.assertEqual(
            hashlib.md5(b"hello").hexdigest(),
            "5d41402abc4b2a76b9719d911017c592",
        )
    def test_published_ja3_reference_vector(self):
        # Create a small object representing the published JA3
example.
        # SimpleNamespace lets us supply the fields expected by our
function.
        hello = SimpleNamespace(
            version=769,
            ciphers=[
                47, 53, 5, 10, 49161, 49162,
                49171, 49172, 50, 56, 19, 4,
            ],
            ext=[
                SimpleNamespace(type=0),
                SimpleNamespace(type=10, groups=[23, 24, 25]),
                SimpleNamespace(type=11, ecpl=[0]),
            ],
        )

        ja3_string, fingerprint = calculate_ja3(hello)

        expected_string = (
            "769,"
            "47-53-5-10-49161-49162-49171-49172-50-56-19-4,"
            "0-10-11,"
```

```
            "23-24-25,"
            "0"
        )

        expected_fingerprint =
"ada70206e40642a3e4461f35503241d5"

        self.assertEqual(ja3_string, expected_string)
        self.assertEqual(fingerprint, expected_fingerprint)

if __name__ == "__main__":
    unittest.main()
```

above code is test_fingerprints.py

```
from scapy.all import load_layer, sniff
from scapy.sessions import TCPSession

load_layer("tls")

from scapy.layers.tls.handshake import (
    TLSClientHello,
    TLSServerHello,
)

pcap_file = "captures/first_https.pcap"

packets = sniff(
    offline=pcap_file,
    session=TCPSession,
    store=True,
)

client_count = sum(
    packet.haslayer(TLSClientHello)
    for packet in packets
)

server_count = sum(
    packet.haslayer(TLSServerHello)
    for packet in packets
)

print(f"Capture: {pcap_file}")
print(f"Packets returned by Scapy: {len(packets)}")
```

print(f"ClientHello messages: {client_count}")
print(f"ServerHello messages: {server_count}")

above code is check_reassembly.py
now tell me what else still we need to do
why are you losing memory of what we had done from initially,remember these code files atleast upto end of this conversation

Memory updated

ChatGPT said:

You said:

yes,while suggesting changes also reason to the point ,so that i could understand clearly

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/inspect_tls.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Packets returned by Scapy (TCP session mode): 16

ClientHello #1
  Source: 172.23.55.87
  Destination: 104.20.23.154
  Legacy version: 771
  Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
  Extension IDs: [0, 11, 10, 16, 22, 23, 49, 13, 43, 45, 51, 21]
  Supported groups: [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
  EC point formats: [0, 1, 2]

ServerHello #1
  Source: 104.20.23.154
  Destination: 172.23.55.87
  Legacy version: 771
  Selected cipher: 4866

```
  Extension IDs:       [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 16

ClientHello #1
  Source:       172.23.55.87
  Destination:  104.20.23.154
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-16-22-23-49-13-43-45-51-21,29-23-30-25-24-256-257-258-
259-260,0-1-2
  JA3 MD5:      0149f47eabf9a20d0893e2a44e5a6323

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python src/ja3s.py captures/first_https.pcap
Capture file: captures/first_https.pcap
Total packets: 16

ServerHello #1
  Source:       104.20.23.154
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
  JA3S MD5:     907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$ python -m unittest discover -s tests -t . -v
test_grease_values_are_removed
(tests.test_fingerprints.FingerprintTests.test_grease_values_are_remo
ved) ... ok
test_ja3_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3_fingerprint) ... ok
test_ja3s_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3s_fingerprint) ... ok
```

```
test_md5_known_value
(tests.test_fingerprints.FingerprintTests.test_md5_known_value) ... ok
test_published_ja3_reference_vector
(tests.test_fingerprints.FingerprintTests.test_published_ja3_reference
_vector) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.032s

OK
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/CN/CN_PROJECT2
$
```

Worked for 27s >

Analysis paused

ChatGPT said:

‹/›

You said:

all 6tests passed

ChatGPT said:

You said:

but,before that we need to change code in our existing files as they contain some hardcoded values of our first_https.pcap instead of any capture

ChatGPT said:

You said:

if we are putting a baseline pcap file in test_fingerprints then why can't we just put an input like other files and when we don't give that input pcap file raise an error ,so that we don't get baseline result everytime when we forget the input of new pcap file ,i think it is best practice as other files also follow the same and also give the

importance of each file ,if any file is unnecesaary ,will delete it instead of unnecessary code base

Memory updated

ChatGPT said:

You said:

yeah done, let's start capturing traffic from different sources,also one of our senior's used
cloudflare_run1.pcap
curl.pcap
python_requests.pcap
custom_client.pcap
chrome.pcap
chrome_run2.pcap
stress_test.pcap
in their report
don't use same names but ideas will work due to plaigairism and also tell what methods they used as i couldn't understand from filenames itself

Worked for 1m 22s

ChatGPT said:

You said:

(.venv) busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ .venv/bin/python src/inspect_tls.py captures/capture_02_openssl_cli_example_com.pcap
Capture file: captures/capture_02_openssl_cli_example_com.pcap
Packets returned by Scapy (TCP session mode): 13

ClientHello #1
  Source: 172.23.55.87
  Destination: 104.20.23.154
  Legacy version: 771
  Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
  Extension IDs: [0, 11, 10, 35, 22, 23, 13, 43, 45, 51]

```
  Supported groups:    [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
  EC point formats:    [0, 1, 2]

ServerHello #1
  Source:              104.20.23.154
  Destination:         172.23.55.87
  Legacy version:      771
  Selected cipher:     4866
  Extension IDs:       [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ .venv/bin/python src/ja3.py
captures/capture_02_openssl_cli_example_com.pcap
Capture file: captures/capture_02_openssl_cli_example_com.pcap
Total packets: 13

ClientHello #1
  Source:       172.23.55.87
  Destination:  104.20.23.154
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-35-22-23-13-43-45-51,29-23-30-25-24-256-257-258-259-260,0-
1-2
  JA3 MD5:      a3afc2c46ba4a7d7fbe1cfb7a3031c2f

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ .venv/bin/python src/ja3s.py
captures/capture_02_openssl_cli_example_com.pcap
Capture file: captures/capture_02_openssl_cli_example_com.pcap
Total packets: 13

ServerHello #1
  Source:       104.20.23.154
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
  JA3S MD5:     907bf3ecef1c987c889946b737b43de8
```

ServerHello messages processed: 1

Memory updated

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ curl -V
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.16
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ openssl version -a
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
built on: Wed Sep 16 18:47:02 2026 UTC
platform: debian-amd64
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall -fzero-call-used-regs=used-gpr -DOPENSSL_TLS_SECURITY_LEVEL=2 -Wa,--noexecstack -g -O2 -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -ffile-prefix-map=/build/openssl-yMWwKu/openssl-3.0.13=. -fstack-protector-strong -fstack-clash-protection -Wformat -Werror=format-security -fcf-protection -fdebug-prefix-map=/build/openssl-yMWwKu/openssl-3.0.13=/usr/src/openssl-3.0.13-0ubuntu3.16 -DOPENSSL_USE_NODELETE -DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DNDEBUG -Wdate-time -D_FORTIFY_SOURCE=3
OPENSSLDIR: "/usr/lib/ssl"
ENGINESDIR: "/usr/lib/x86_64-linux-gnu/engines-3"
MODULESDIR: "/usr/lib/x86_64-linux-gnu/ossl-modules"
Seeding source: os-specific
CPUINFO:

OPENSSL_ia32cap=0xfffa32235f8bffff:0x184007a4219c27ab

but those extension lists might also contain GREASE values know because these are random, extension lists from same openssl will also have different ja3 string?

Worked for 46s

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ .venv/bin/python -m unittest discover -s tests -t . -v
test_grease_values_are_removed
(tests.test_fingerprints.FingerprintTests.test_grease_values_are_removed) ... ok
test_ja3_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3_fingerprint) ... ok
test_ja3_ignores_grease_values
(tests.test_fingerprints.FingerprintTests.test_ja3_ignores_grease_values) ... ok
test_ja3s_fingerprint
(tests.test_fingerprints.FingerprintTests.test_ja3s_fingerprint) ... ok
test_ja3s_reference_vector
(tests.test_fingerprints.FingerprintTests.test_ja3s_reference_vector) ... ok
test_md5_known_value
(tests.test_fingerprints.FingerprintTests.test_md5_known_value) ... ok
test_published_ja3_reference_vector
(tests.test_fingerprints.FingerprintTests.test_published_ja3_reference_vector) ... ok

----
Ran 7 tests in 0.064s

OK

Memory updated

ChatGPT said:

You said:

busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ .venv/bin/python src/ja3.py captures/capture_03_openssl_repeatibilty_example_com.pcap
Capture file: captures/capture_03_openssl_repeatibilty_example_com.pcap
Total packets: 42

ClientHello #1
  Source: 172.23.55.87
  Destination: 172.66.147.243
  JA3 string: 771,4866-4867-4865-49196-49200-159-52393-52392-52394-49195-49199-158-49188-49192-107-49187-49191-103-49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-10-35-22-23-13-43-45-51,29-23-30-25-24-256-257-258-259-260,0-1-2
  JA3 MD5:   a3afc2c46ba4a7d7fbe1cfb7a3031c2f

ClientHello #2
  Source: 172.23.55.87
  Destination: 104.20.23.154
  JA3 string: 771,4866-4867-4865-49196-49200-159-52393-52392-52394-49195-49199-158-49188-49192-107-49187-49191-103-49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-10-35-22-23-13-43-45-51,29-23-30-25-24-256-257-258-259-260,0-1-2
  JA3 MD5:   a3afc2c46ba4a7d7fbe1cfb7a3031c2f

ClientHello #3
  Source: 172.23.55.87
  Destination: 104.20.23.154
  JA3 string: 771,4866-4867-4865-49196-49200-159-52393-52392-52394-49195-49199-158-49188-49192-107-49187-49191-103-49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-10-35-22-23-13-43-45-51,29-23-30-25-24-256-257-258-259-260,0-1-2
  JA3 MD5:   a3afc2c46ba4a7d7fbe1cfb7a3031c2f

ClientHello messages processed: 3
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ .venv/bin/python src/ja3s.py captures/capture_03_openssl_repeatibilty_example_com.pcap
Capture file: captures/capture_03_openssl_repeatibilty_example_com.pcap

Total packets: 42

ServerHello #1
Source: 172.66.147.243
Destination: 172.23.55.87
JA3S string: 771,4866,51-43
JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello #2
Source: 104.20.23.154
Destination: 172.23.55.87
JA3S string: 771,4866,51-43
JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello #3
Source: 104.20.23.154
Destination: 172.23.55.87
JA3S string: 771,4866,51-43
JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 3

Worked for 19s >

ChatGPT said:

You said:

busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ .venv/bin/python src/inspect_tls.py captures/capture_03_openssl_repeatibilty_example_com.pcap
Capture file: captures/capture_03_openssl_repeatibilty_example_com.pcap
Packets returned by Scapy (TCP session mode): 42

ClientHello #1
Source: 172.23.55.87
Destination: 172.66.147.243
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
Extension IDs: [0, 11, 10, 35, 22, 23, 13, 43, 45, 51]
Supported groups: [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]

EC point formats: [0, 1, 2]

ServerHello #1
Source: 172.66.147.243
Destination: 172.23.55.87
Legacy version: 771
Selected cipher: 4866
Extension IDs: [51, 43]

ClientHello #2
Source: 172.23.55.87
Destination: 104.20.23.154
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
Extension IDs: [0, 11, 10, 35, 22, 23, 13, 43, 45, 51]
Supported groups: [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
EC point formats: [0, 1, 2]

ServerHello #2
Source: 104.20.23.154
Destination: 172.23.55.87
Legacy version: 771
Selected cipher: 4866
Extension IDs: [51, 43]

ClientHello #3
Source: 172.23.55.87
Destination: 104.20.23.154
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
Extension IDs: [0, 11, 10, 35, 22, 23, 13, 43, 45, 51]
Supported groups: [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
EC point formats: [0, 1, 2]

ServerHello #3
Source: 104.20.23.154
Destination: 172.23.55.87
Legacy version: 771
Selected cipher: 4866

Extension IDs: [51, 43]

--- Summary ---
ClientHello messages parsed: 3
ServerHello messages parsed: 3

i think repeatibility is no longer problem,producing same thing, why can't we move on to what project wants,also if curl uses openssl library and openssl also uses same openssl library then we can say that both use different configuration of same library?if it is the case then why don't we note down or if there is a curl library then try it otherwise let's move on

Worked for 46s >

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/inspect_tls.py captures/capture_04_python_ssl_example_com.pcap
Capture file: captures/capture_04_python_ssl_example_com.pcap
Packets returned by Scapy (TCP session mode): 10

ClientHello #1
Source: 172.23.55.87
Destination: 172.66.147.243
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191, 103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47, 255]
Extension IDs: [0, 11, 10, 35, 22, 23, 13, 43, 45, 51, 21]
Supported groups: [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
EC point formats: [0, 1, 2]

ServerHello #1
Source: 172.66.147.243
Destination: 172.23.55.87
Legacy version: 771
Selected cipher: 4866
Extension IDs: [51, 43]

```
--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3.py
captures/capture_04_python_ssl_example_com.pcap
pythoCapture file:
captures/capture_04_python_ssl_example_com.pcap
Total packets: 10

ClientHello #1
  Source:       172.23.55.87
  Destination:  172.66.147.243
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-35-22-23-13-43-45-51-21,29-23-30-25-24-256-257-258-259-
260,0-1-2
  JA3 MD5:      d39e1be3241d516b1f714bd47c2bc968

ClientHello messages processed: 1
n(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3s.py
captures/capture_4_python_ssl_example_com.pcap
Capture file: captures/capture_04_python_ssl_example_com.pcap
Total packets: 10

ServerHello #1
  Source:       172.66.147.243
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
  JA3S MD5:     907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$
```

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python -c "import ssl;print(ssl.OPENSSL_VERSION)"
OpenSSL 3.0.13 30 Jan 2024

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/inspect_tls.py
captures/capture_05_python_requests_example_com.pcap
Capture file:
captures/capture_05_python_requests_example_com.pcap
Packets returned by Scapy (TCP session mode): 15

ClientHello #1
  Source:             172.23.55.87
  Destination:        104.20.23.154
  Legacy version:     771
  Cipher suites:      [4866, 4867, 4865, 49196, 49200, 159, 52393,
52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191,
103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47,
255]
  Extension IDs:      [0, 11, 10, 16, 22, 23, 49, 13, 43, 45, 51, 21]
  Supported groups:   [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
  EC point formats:   [0, 1, 2]

ServerHello #1
  Source:             104.20.23.154
  Destination:        172.23.55.87
  Legacy version:     771
  Selected cipher:    4866
  Extension IDs:      [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3.py
captures/capture_05_python_requests_example_com.pcap
Capture file:
captures/capture_05_python_requests_example_com.pcap
Total packets: 15
```

ClientHello #1
Source: 172.23.55.87
Destination: 104.20.23.154
JA3 string: 771,4866-4867-4865-49196-49200-159-52393-52392-52394-49195-49199-158-49188-49192-107-49187-49191-103-49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-10-16-22-23-49-13-43-45-51-21,29-23-30-25-24-256-257-258-259-260,0-1-2
JA3 MD5: 0149f47eabf9a20d0893e2a44e5a6323

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/ja3s.py captures/capture_05_python_requests_example_com.pcap
Capture file: captures/capture_05_python_requests_example_com.pcap
Total packets: 15

ServerHello #1
Source: 104.20.23.154
Destination: 172.23.55.87
JA3S string: 771,4866,51-43
JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1

ChatGPT said:

You said:

see since all use same TLS backend engines then why can't we delete all captures and let use different engines for each of them so we can identify upon fingerprints that they used diffrent engines atleast

Worked for 1m 58s >

ChatGPT said:

You said:

wait before doing next capture,why can't you check whether we are moving in correct direction according to problem statement

Project Description: Build a tool that passively captures TLS ClientHello and ServerHello messages and computes JA3/JA3S fingerprints (with JA4/JA4S as a stretch goal) to identify the client or server application/library generating the traffic. Curate a small reference database and demonstrate distinguishing real clients (browsers, curl, custom scripts) purely from their handshake fingerprint.
Tools/Technologies: : libpcap/Scapy (or raw sockets) for capture; fingerprint computation implemented per the JA3/JA4 specifications, eBPF/XDP, Packet/Flow Generators like TRex, KV-stores (Redis, Valkey).
Expected Outcome: Demonstrate the working JA3/JA3S fingerprint extractor, validated against published reference JA3 hashes for known clients; A curated database correctly identifying at least 5 distinct clients/tools by fingerprint alone. Demonstrate distinguishing, e.g., curl vs. a browser vs. a custom TLS client on the wire.
Understanding on the fingerprinting's role in security monitoring (malware C2 detection, client identification) and its limitations, including fingerprint randomization in modern browsers as an evasion technique.

Worked for 2m 13s >

ChatGPT said:

You said:

i agree with you, let's capture with remaining methods Go,Chrome,GnuTLS and custom TLS configuration experiment and then checking chrome's behaviour and i think our repeatibility experiments with the other taken clients also useful because taking only chrome for repeatibility is like just checking fact know

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ go version
Command 'go' not found, but can be installed with:
sudo snap install go   # version 1.27.2, or
sudo apt install golang-go # version 2:1.21~2

```
sudo apt  install gccgo-go   # version 2:1.21~2
See 'snap info go' for additional versions.
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ gnutls-cli --version
Command 'gnutls-cli' not found, but can be installed with:
sudo apt install gnutls-bin
```

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ go version
go version go1.22.2 linux/amd64
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ gnutls-cli --version
gnutls-cli 3.8.3
Copyright (C) 2000-2023 Free Software Foundation, and others
This is free software. It is licensed for use, modification and
redistribution under the terms of the GNU General Public License,
version 3 or later <http://gnu.org/licenses/gpl.html>

Please send bug reports to:  <bugs@gnutls.org>
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ go env GOVERSION GOROOT
go1.22.2
/usr/lib/go-1.22
```

ChatGPT said:

You said:

```
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/inspect_tls.py
captures/capture_06_go_crypto_tls_example_com.pcap
Capture file: captures/capture_06_go_crypto_tls_example_com.pcap
Packets returned by Scapy (TCP session mode): 18

ClientHello #1
```

```
  Source:             172.23.55.87
  Destination:        172.66.147.243
  Legacy version:     771
  Cipher suites:      [49195, 49199, 49196, 49200, 52393, 52392,
49161, 49171, 49162, 49172, 49170, 4865, 4866, 4867]
  Extension IDs:      [0, 5, 10, 11, 13, 65281, 23, 18, 43, 51]
  Supported groups:   [29, 23, 24, 25]
  EC point formats:   [0]

ServerHello #1
  Source:             172.66.147.243
  Destination:        172.23.55.87
  Legacy version:     771
  Selected cipher:    4865
  Extension IDs:      [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3.py
captures/capture_06_go_crypto_tls_example_com.pcap
Capture file: captures/capture_06_go_crypto_tls_example_com.pcap
Total packets: 18

ClientHello #1
  Source:       172.23.55.87
  Destination:  172.66.147.243
  JA3 string:   771,49195-49199-49196-49200-52393-52392-49161-
49171-49162-49172-49170-4865-4866-4867,0-5-10-11-13-65281-
23-18-43-51,29-23-24-25,0
  JA3 MD5:      88476a293aeb6f5dc7a3630d391c2980

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3s.py
captures/capture_06_go_crypto_tls_example_com.pcap
Capture file: captures/capture_06_go_crypto_tls_example_com.pcap
Total packets: 18

ServerHello #1
  Source:       172.66.147.243
  Destination:  172.23.55.87
```

JA3S string: 771,4865,51-43
JA3S MD5: eb1d94daa7e0344597e756a1fb6e7054

ServerHello messages processed: 1

Worked for 52s >

ChatGPT said:

You said:

(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/inspect_tls.py captures/capture_07_gnutls_cli_example_com.pcap
Capture file: captures/capture_07_gnutls_cli_example_com.pcap
Packets returned by Scapy (TCP session mode): 15

ClientHello #1
Source: 172.23.55.87
Destination: 172.66.147.243
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 4868, 49196, 52393, 49325, 49162, 49195, 49324, 49161, 49200, 52392, 49172, 49199, 49171, 157, 49309, 53, 156, 49308, 47, 159, 52394, 49311, 57, 158, 49310, 51]
Extension IDs: [11, 13, 22, 51, 65281, 28, 45, 43, 35, 10, 5, 0, 23]
Supported groups: [23, 24, 25, 29, 30, 256, 257, 258, 259, 260]
EC point formats: [0]

ServerHello #1
Source: 172.66.147.243
Destination: 172.23.55.87
Legacy version: 771
Selected cipher: 4866
Extension IDs: [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/ja3.py captures/capture_07_gnutls_cli_example_com.pcap
Capture file: captures/capture_07_gnutls_cli_example_com.pcap

Total packets: 15

ClientHello #1
Source: 172.23.55.87
Destination: 172.66.147.243
JA3 string: 771,4866-4867-4865-4868-49196-52393-49325-49162-49195-49324-49161-49200-52392-49172-49199-49171-157-49309-53-156-49308-47-159-52394-49311-57-158-49310-51,11-13-22-51-65281-28-45-43-35-10-5-0-23,23-24-25-29-30-256-257-258-259-260,0
JA3 MD5: 75484e4f01a386b961233d4238ccac89

ClientHello messages processed: 1
PROJECT2$ python src/ja3s.py captures/capture_07_gnutls_cli_example_com.pcap
Capture file: captures/capture_07_gnutls_cli_example_com.pcap
Total packets: 15

ServerHello #1
Source: 172.66.147.243
Destination: 172.23.55.87
JA3S string: 771,4866,51-43
JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1

Worked for 36s >

ChatGPT said:

You said:

(.venv) busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/inspect_tls.py captures/capture_08_openssl_alpn_example_com.pcap
Capture file: captures/capture_08_openssl_alpn_example_com.pcap
Packets returned by Scapy (TCP session mode): 12

ClientHello #1
Source: 172.23.55.87
Destination: 104.20.23.154
Legacy version: 771
Cipher suites: [4866, 4867, 4865, 49196, 49200, 159, 52393, 52392, 52394, 49195, 49199, 158, 49188, 49192, 107, 49187, 49191,

```
103, 49162, 49172, 57, 49161, 49171, 51, 157, 156, 61, 60, 53, 47,
255]
  Extension IDs:       [0, 11, 10, 35, 16, 22, 23, 13, 43, 45, 51]
  Supported groups:    [29, 23, 30, 25, 24, 256, 257, 258, 259, 260]
  EC point formats:    [0, 1, 2]

ServerHello #1
  Source:              104.20.23.154
  Destination:         172.23.55.87
  Legacy version:      771
  Selected cipher:     4866
  Extension IDs:       [51, 43]

--- Summary ---
ClientHello messages parsed: 1
ServerHello messages parsed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3.py
captures/capture_08_openssl_alpn_example_com.pcap
Capture file: captures/capture_08_openssl_alpn_example_com.pcap
Total packets: 12

ClientHello #1
  Source:       172.23.55.87
  Destination:  104.20.23.154
  JA3 string:   771,4866-4867-4865-49196-49200-159-52393-52392-
52394-49195-49199-158-49188-49192-107-49187-49191-103-
49162-49172-57-49161-49171-51-157-156-61-60-53-47-255,0-11-
10-35-16-22-23-13-43-45-51,29-23-30-25-24-256-257-258-259-
260,0-1-2
  JA3 MD5:      5a1edc7f170af1014fc65c994878e63c

ClientHello messages processed: 1
(.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PRO
JECT2$ python src/ja3s.py
captures/capture_08_openssl_alpn_example_com.pcap
Capture file: captures/capture_08_openssl_alpn_example_com.pcap
Total packets: 12

ServerHello #1
  Source:       104.20.23.154
  Destination:  172.23.55.87
  JA3S string:  771,4866,51-43
```

JA3S MD5: 907bf3ecef1c987c889946b737b43de8

ServerHello messages processed: 1

Worked for 1m 34s >

ChatGPT said:

You said:

PROJECT2$ python src/inspect_tls.py
captures/capture_09_chrome_example_com.pcapng
WARNING: <class 'scapy.sessions.TCPSession'> processing failed with 'unpack requires a buffer of 1 bytes'. Skipping.
WARNING: <class 'scapy.sessions.TCPSession'> processing failed with 'unpack requires a buffer of 1 bytes'. Skipping.
WARNING: more <class 'scapy.sessions.TCPSession'> processing failed with 'unpack requires a buffer of 1 bytes'. Skipping.
Capture file: captures/capture_09_chrome_example_com.pcapng
Packets returned by Scapy (TCP session mode): 399

ClientHello #1
Source: 10.7.22.45
Destination: 142.251.153.119
Legacy version: 771
Cipher suites: [27242, 4865, 4866, 4867, 49195, 49199, 49196, 49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
Extension IDs: [60138, 35, 18, 51, 27, 51764, 45, 23, 16, 10, 43, 11, 5, 65037, 13, 17613, 65281, 0, 14906]
Supported groups: [6682, 4588, 29, 23, 24]
EC point formats: [0]

ClientHello #2
Source: 10.7.22.45
Destination: 142.251.154.119
Legacy version: 771
Cipher suites: [23130, 4865, 4866, 4867, 49195, 49199, 49196, 49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
Extension IDs: [14906, 0, 17613, 45, 11, 10, 23, 65037, 13, 18, 27, 5, 16, 51764, 51, 65281, 43, 35, 19018, 41]
Supported groups: [31354, 4588, 29, 23, 24]
EC point formats: [0]

ServerHello #1
Source: 142.251.153.119

Destination: 10.7.22.45
Legacy version: 771
Selected cipher: 4865
Extension IDs: [51, 43]

ServerHello #2
Source: 142.251.154.119
Destination: 10.7.22.45
Legacy version: 771
Selected cipher: 4865
Extension IDs: [41, 51, 43]

ClientHello #3
Source: 10.7.22.45
Destination: 34.96.128.111
Legacy version: 771
Cipher suites: [64250, 4865, 4866, 4867, 49195, 49199, 49196, 49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
Extension IDs: [6682, 10, 13, 23, 65281, 51764, 45, 51, 16, 11, 5, 18, 0, 27, 65037, 17613, 35, 43, 60138]
Supported groups: [60138, 4588, 29, 23, 24]
EC point formats: [0]

ServerHello #3
Source: 34.96.128.111
Destination: 10.7.22.45
Legacy version: 771
Selected cipher: 49199
Extension IDs: [35, 65281, 16]

ClientHello #4
Source: 10.7.22.45
Destination: 192.178.177.94
Legacy version: 771
Cipher suites: [43690, 4865, 4866, 4867, 49195, 49199, 49196, 49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
Extension IDs: [14906, 43, 35, 0, 13, 27, 65281, 51, 51764, 11, 23, 16, 65037, 10, 5, 45, 17613, 18, 39578]
Supported groups: [23130, 4588, 29, 23, 24]
EC point formats: [0]

ServerHello #4
Source: 192.178.177.94
Destination: 10.7.22.45
Legacy version: 771

```
  Selected cipher:     4865
  Extension IDs:       [51, 43]

ClientHello #5
  Source:              10.7.22.45
  Destination:         104.20.23.154
  Legacy version:      771
  Cipher suites:       [60138, 4865, 4866, 4867, 49195, 49199, 49196,
49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
  Extension IDs:       [56026, 18, 23, 51764, 0, 16, 43, 27, 51, 45,
65281, 17613, 10, 35, 65037, 13, 11, 5, 43690]
  Supported groups:    [23130, 4588, 29, 23, 24]
  EC point formats:    [0]

ServerHello #5
  Source:              104.20.23.154
  Destination:         10.7.22.45
  Legacy version:      771
  Selected cipher:     4865
  Extension IDs:       [51, 43]

ClientHello #6
  Source:              10.7.22.45
  Destination:         172.66.147.243
  Legacy version:      771
  Cipher suites:       [4866, 4865, 49196, 49195, 49200, 49199, 49188,
49187, 49192, 49191, 49162, 49161, 49172, 49171, 157, 156, 61, 60,
53, 47]
  Extension IDs:       [0, 43, 13, 35, 10, 11, 51, 49, 23, 65281, 45]
  Supported groups:    [29, 23, 24]
  EC point formats:    [0]

ServerHello #6
  Source:              172.66.147.243
  Destination:         10.7.22.45
  Legacy version:      771
  Selected cipher:     4866
  Extension IDs:       [51, 43]

--- Summary ---
ClientHello messages parsed: 6
ServerHello messages parsed: 6
```

see the error

Worked for 59s >

ChatGPT said:

You said:

see i'm using four tabs in my college account chrome in which one tab is talking with you,whereas for disabling QUIC i used my google chrome for personal account and in that settings of QUIC is diabled and i opened incognito in that personal account google chrome desktop app,do i need to capture again with closing my chrome with which i'm chatting with you untill capture done?

ChatGPT said:

You said:

.venv)
busabhavaram@BhavaRam:/mnt/c/Users/busab/SEM5/CN/CN_PROJECT2$ python src/inspect_tls.py captures/capture_09_chrome_example_com.pcapng
Capture file: captures/capture_09_chrome_example_com.pcapng
Packets returned by Scapy (TCP session mode): 107

ClientHello #1
  Source: 10.7.22.45
  Destination: 142.251.153.119
  Legacy version: 771
  Cipher suites: [35466, 4865, 4866, 4867, 49195, 49199, 49196, 49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
  Extension IDs: [56026, 51764, 27, 65281, 16, 5, 43, 17613, 13, 51, 45, 18, 0, 11, 10, 23, 35, 65037, 6682]
  Supported groups: [56026, 4588, 29, 23, 24]
  EC point formats: [0]

ServerHello #1
  Source: 142.251.153.119
  Destination: 10.7.22.45
  Legacy version: 771
  Selected cipher: 4865
  Extension IDs: [51, 43]

ClientHello #2
  Source: 10.7.22.45

```
  Destination:        35.190.80.143
  Legacy version:     771
  Cipher suites:      [64250, 4865, 4866, 4867, 49195, 49199, 49196,
49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
  Extension IDs:      [60138, 51764, 45, 17613, 27, 51, 13, 5, 0, 18, 35,
65281, 10, 65037, 16, 43, 23, 11, 47802, 41]
  Supported groups:   [10794, 4588, 29, 23, 24]
  EC point formats:   [0]

ServerHello #2
  Source:             35.190.80.143
  Destination:        10.7.22.45
  Legacy version:     771
  Selected cipher:    4865
  Extension IDs:      [41, 51, 43]

ClientHello #3
  Source:             10.7.22.45
  Destination:        142.250.118.101
  Legacy version:     771
  Cipher suites:      [10794, 4865, 4866, 4867, 49195, 49199, 49196,
49200, 52393, 52392, 49171, 49172, 156, 157, 47, 53]
  Extension IDs:      [43690, 13, 65281, 18, 51764, 51, 65037, 45, 27,
35, 10, 17613, 16, 0, 5, 23, 43, 11, 2570, 41]
  Supported groups:   [14906, 4588, 29, 23, 24]
  EC point formats:   [0]

ServerHello #3
  Source:             142.250.118.101
  Destination:        10.7.22.45
  Legacy version:     771
  Selected cipher:    4865
  Extension IDs:      [41, 51, 43]

--- Summary ---
ClientHello messages parsed: 3
ServerHello messages parsed: 3
```

ChatGPT said: